Heurísticas de Erdoğan et al. (2012): del TSPPD-H al TRPPD-H_3

Pseudocódigos y diferencias entre el artículo y este proyecto

Taller de investigación

Noviembre de 2026

Erdoğan, Battarra, Laporte y Vigo (2012) proponen una heurística de 2 fases y tres metaheurísticas (TS, ILS e ITS) para el TSPPD-H, cada una con una evaluación *exacta* y otra *heurística* de los vecinos. Este proyecto adapta cinco de esos métodos al TRPPD-H_3: la heurística de 2 fases, ILS exacto, ILS heurístico, ITS exacto e ITS heurístico. Para cada uno, este documento explica qué hace el artículo, qué cambió en el proyecto y por qué, y da el pseudocódigo de la versión adaptada.

Convención. En los pseudocódigos, lo que está en carmesí es distinto del artículo. Debajo de cada algoritmo, un cuadro compara esas líneas con su versión original. Para no mezclar numeraciones, los algoritmos de este documento van numerados $1, 2, \ldots$ y sus tablas se llaman *cuadros*; "Algoritmo 4.2", "Tabla 5", "Fig. 1" y "fórmula (14)" se refieren siempre al artículo.

Índice

1. Los dos problemas **1**

2. Resumen de las diferencias **2**

3. Notación **3**

4. Exacto y heurístico **4**
4.1. Evaluación exacta 5
4.2. Evaluación heurística 6

5. Heurística de 2 fases **8**

6. Vecindario y perturbación **9**

7. ILS exacto e ILS heurístico **10**

8. ITS exacto e ITS heurístico **12**

9. Ejecución en los dos sentidos **14**

10. Resultados **15**

1. Los dos problemas

En los dos problemas un vehículo con una sola puerta trasera sale del depósito con todas las entregas, visita una vez a cada cliente $i \in V_c$, le entrega α_i unidades y le recoge β_i, y vuelve al depósito sin superar la capacidad Q. Bajo la Política 3 de Battarra et al. (2010), en cada cliente se decide si las recogidas quedan junto a la puerta (Política 1, $s_i = 1$) o al fondo (Política 2, $s_i = 0$), lo que obliga a retirar y recargar las entregas que quedan. Lo que cambia es el objetivo:

- **TSPPD-H** (artículo): minimizar el costo de viaje más el costo de manipulación,

$$z^{\mathrm{TSP}}(\pi, s) = \sum_{k=1}^{n+1} c_{\pi_{k-1}\pi_k} + H(\pi, s), \qquad \pi_0 = \pi_{n+1} = 0,$$

  donde H suma h_a por cada unidad de entrega y h_b por cada unidad de recogida que se mueve en una operación adicional. Las operaciones obligatorias (descargar α_i y cargar β_i) son constantes y quedan fuera de la evaluación.

- **TRPPD-H$_3$** (proyecto): minimizar la suma de los instantes D_i en que cada cliente termina de recibir su entrega. Cada duración atrasa a todos los clientes que aún no terminan; en la posición k pesa $W_k = n - k + 1$ si ocurre antes de completar la entrega a π_k, y $W_k - 1$ si ocurre después:

$$\sum_{i \in V_c} D_i = \sum_{k=1}^{n} \Big[W_k \big(c_{\pi_{k-1}\pi_k} + u_a \alpha_{\pi_k} + u_b z_k \big) + (W_k - 1) \big(\ell_b (z_k + \beta_{\pi_k}) + q_{\pi_k} \big) \Big],$$

  con z_k las recogidas junto a la puerta al llegar a π_k y $q_{\pi_k} = h_a \sum_{m>k} \alpha_{\pi_m}$ si en π_k se aplica la Política 2 (0 si no).

Cuadro 1: Cómo se mide cada componente del costo.

Elemento	Erdoğan et al. (2012) · TSPPD-H	Este proyecto · TRPPD-H$_3$
Objetivo	viaje + manipulación	$\sum_i D_i$ (latencia)
Arco (π_{k-1}, π_k)	$c_{\pi_{k-1}\pi_k}$	$W_k \, c_{\pi_{k-1}\pi_k}$
Regreso al depósito	$c_{\pi_n 0}$	no cuenta: ya no atrasa a ningún cliente
Operaciones obligatorias	constantes, fuera de la evaluación	$W_k \, u_a \alpha_{\pi_k} + (W_k - 1) \, \ell_b \beta_{\pi_k}$: dependen de la posición, entran en la parte fija $F(\pi)$
Recogidas que estorban en π_k	$h_b z_k$ (si $\alpha_{\pi_k} > 0$)	$\big(u_b W_k + \ell_b (W_k - 1)\big) z_k$: la descarga ocurre antes de entregar y la recarga después
Política 2 en π_k	$h_a \sum_{m>k} \alpha_{\pi_m}$	$h_a \sum_{m>k} \alpha_{\pi_m} \, (W_k - 1)$
Tiempos de manipulación	h_a, h_b (en los experimentos, $h_a = h_b = h$)	u_a, ℓ_a, u_b, ℓ_b por separado ($= h/2$ en las instancias); $h_a = u_a + \ell_a$, $h_b = u_b + \ell_b$
Sentido de la ruta	con c simétrica, la ruta y su inversa tienen el mismo costo de viaje	la ruta y su inversa tienen distinta latencia

Tres consecuencias guían toda la adaptación:

1. **El DP sigue siendo exacto.** Con la ruta fija, los pesos W_k dependen solo de la posición, así que el costo de manipulación se sigue separando en tramos entre dos Políticas 2, y la recursión (1)–(2) del artículo vale con costos p_{ij} ponderados. Se comprobó contra la enumeración de las 2^{10} políticas en 120 rutas con $n = 10$.

2. **El viaje ya no se actualiza en $O(1)$.** En el TSPPD-H el costo de viaje de un vecino se obtiene sumando los arcos nuevos y restando los quitados. En el TRPPD-H$_3$ un movimiento cambia la posición, y por lo tanto el peso, de todos los clientes que siguen; la parte fija $F(\pi)$ se recalcula en $O(n)$. El orden de la evaluación completa no cambia: $O(n^2)$ exacta y $O(n)$ heurística.

3. **La rotación y el sentido pesan mucho más.** En el TSPPD-H, mover el depósito dentro del ciclo solo cambia los arcos que lo rodean, y con c simétrica recorrer el ciclo al revés no cambia el largo. En el TRPPD-H$_3$, elegir con qué cliente empezar y en qué sentido recorrer decide qué clientes se atienden primero, y con eso el peso de todos los arcos.

2. Resumen de las diferencias

El Cuadro 2 reúne las diferencias de los cinco métodos; las secciones siguientes las desarrollan método por método.

Cuadro 2: Diferencias entre el artículo y este proyecto, por componente.

Elemento	Erdoğan et al. (2012) · TSPPD-H	Este proyecto · TRPPD-H$_3$
Evaluación de una ruta fija		
Costo de la ruta	viaje + manipulación	$\sum_i D_i$, con pesos W_k (Cuadro 1)
Evaluación exacta	Algoritmo 2.1 + DP (1)–(2), $O(n^2)$	misma estructura, con p_{ij} ponderados (Algoritmos 1 y 2)
Evaluación heurística	umbral 2 (recogidas a bordo) con (13)–(14), $O(n)$	misma regla, sin volver a deducirla con los pesos W_k; el costo de las decisiones sí se mide con ellos (Algoritmos 3 y 4)
Heurística de 2 fases		
Fase 1	Lin–Kernighan (Concorde)	vecino más cercano desde los $n+1$ nodos, 2-opt y Or-opt
Fase 2	reubicar el depósito: cambian los arcos que lo rodean y la manipulación	reubicar el depósito en los dos sentidos: cambia la latencia de todo el recorrido
Búsqueda		
Vecindario	relocate y 2-opt (swap descartado, Fig. 3)	igual; invertir toda la ruta es un vecino nuevo
Perturbación	$N_{rand} = 0{,}1\,\lvert V_c\rvert$ movimientos al azar	igual, solo factibles, con semilla
ILS	Algoritmo 4.2, $N_{iter} = 200$	igual
ITS	Algoritmos 4.3 y 4.1, $N^* = 14$, lista $0{,}5\,\lvert V_c\rvert$	igual; cambian el atributo tabú del relocate y la aspiración
TS sola	TS-1dir y TS-2dir (el mejor método del artículo)	no se implementó
Sentidos	1dir o 2dir	siempre 2dir
Experimentos		
Instancias	Battarra et al. ($\lvert V_c\rvert \le 25$) y 100 nuevas de 20 a 200 clientes, $h\,\lvert V_c\rvert = 20$	270: $n = 5$–25 (Battarra) y $n = 30$–60 (Erdoğan), $h \in \{0{,}1;\ 0{,}5;\ 1\}$
Corridas	una por método y sentido	10 réplicas con semillas $1, \ldots, 10$
Referencia	*Best*: lo mejor de todos los métodos	BKS: lo mejor de Gurobi (3600 s) y de todas las réplicas
Implementación	C, Intel Core 2 Quad 2,83 GHz	JavaScript (Node.js); traducción a Python con los mismos resultados

3. Notación

- Una ruta es $\pi = (\pi_1, \ldots, \pi_n)$, con los clientes en el orden de visita; $\pi_0 = 0$ es el depósito.
- En la posición k, una operación anterior a completar la entrega retrasa al cliente π_k y a los siguientes: pesa $W_k = n - k + 1$ en $\sum_i D_i$. Una operación posterior pesa $W_k - 1$.
- $h_a = u_a + \ell_a$ y $h_b = u_b + \ell_b$. La política del cliente i es $s_i = 1$ (Política 1) o $s_i = 0$ (Política 2).
- La carga a bordo no depende de las políticas: π respeta Q si $\sum_{r \in V_c} \alpha_r + \sum_{m=1}^{k} (\beta_{\pi_m} - \alpha_{\pi_m}) \le Q$ para todo $k \in \{0, \ldots, n\}$.
- Vecindario $\mathcal{N}(\pi)$ (Sección 3 del artículo): las rutas que respetan Q y se obtienen con $\textsc{Relocate}(\pi, i, p)$, que lleva el cliente de la posición i a la posición p, o con $\textsc{DosOpt}(\pi, i, j)$, $i < j$, que invierte $\pi_i, \ldots, \pi_j$ (cambia los arcos (π_{i-1}, π_i) y (π_j, π_{j+1}), con $\pi_{n+1} = 0$). Son $n(n-1)$ relocate y $n(n-1)/2$ 2-opt, unas $1{,}5\,n^2$ rutas.
- Atributo tabú $a(\sigma)$ del movimiento que produce σ: el cliente v en un relocate, (v, v); las posiciones (i, j) en un 2-opt.

- Parámetros del artículo: $N_{iter} = 200$, $N_{rand} = \max\{1, \text{round}(0{,}1\,n)\}$, lista tabú de largo $L = \max\{1, \lfloor 0{,}5\,n \rfloor\}$ y $N^* = \lfloor \sqrt{N_{iter}} \rfloor = 14$ en ITS.
- *Exacto* y *heurístico* se refieren a cómo se evalúan los vecinos: con EvalExacta (Algoritmo 2, $O(n^2)$) o con EvalHeur (Algoritmo 4, $O(n)$). La Sección 4 lo explica en detalle.

4. Exacto y heurístico

En el artículo y en el proyecto, "exacto" y "heurístico" **no** describen a la metaheurística: ILS exacto e ITS exacto siguen siendo heurísticas, porque no recorren todas las rutas. Las dos palabras se refieren al *subproblema de manipulación*: dada una ruta fija, decidir la política de cada cliente. Cada vez que la búsqueda mira un vecino, debe resolver ese subproblema para conocer su costo, y puede hacerlo de dos formas.

Cuadro 3: Evaluación exacta y heurística de un vecino.

	Exacta	**Heurística**
Rutina	EvalExacta: Algoritmo 2.1 y DP (1)–(2)	EvalHeur: regla del umbral 2 de la Sección 2.2 del artículo
Qué entrega	las políticas óptimas de la ruta y su costo	unas políticas factibles, no siempre óptimas, y su costo
Costo por vecino	$O(n^2)$	$O(n)$
Vecindario completo	$O(n^4)$	$O(n^3)$
Relación entre valores	$z(\pi)$, el costo exacto	$z_H(\pi) \geq z(\pi)$: cota superior. Se comprobó en 10 800 rutas al azar del proyecto ($n \geq 10$)
Vecino elegido	su valor ya es exacto	se vuelve a evaluar con EvalExacta antes de aceptarlo (Sección 4 del artículo, tercer aspecto común)
Fin de la búsqueda local	ningún vecino mejora	el mejor vecino según la heurística no mejora con su valor exacto, aunque otro vecino sí mejoraría
Solución final	políticas del DP	políticas del DP: la ruta final siempre se reevalúa con EvalExacta

El precio de la versión heurística es elegir a veces un vecino que no es el mejor y detener antes la búsqueda local. En los dos trabajos, el ahorro de tiempo es de un orden de magnitud y la pérdida de calidad es pequeña (Sección 10):

Cuadro 4: Exacto frente a heurístico. Desviación promedio respecto de la referencia (*Best* en el artículo, BKS en el proyecto) y tiempo promedio por instancia.

	Método	Exacto		Heurístico		Ex./Heur.
		Desv.	Tiempo	Desv.	Tiempo	tiempo
Artículo	TS-2dir (Tabla 5)	0,07 %	1000,7 s	2,58 %	117,2 s	8,5
	ILS (Tabla 9)	—	1002,5 s	—	113,7 s	8,8
	ITS (Tabla 9)	—	471,9 s	—	57,4 s	8,2
Proyecto	ILS	0,01 %	43,9 s	0,13 %	6,7 s	6,5
	ITS	0,33 %	3,7 s	0,60 %	0,67 s	5,5

El artículo informa la desviación de las dos evaluaciones solo para TS-2dir; para ILS e ITS da los valores por instancia (Tablas 8–9) y los tiempos. En el proyecto la versión heurística pierde mucho menos que en el artículo (0,1 y 0,3 puntos frente a 2,5). Una explicación posible es el peso de la manipulación: las instancias del artículo se construyeron con $h\,|V_c| = 20$ para que viaje y manipulación pesen parecido, mientras que en las mejores rutas del proyecto la

manipulación adicional es el 9,5 % ($h = 0{,}1$), 21,4 % ($h = 0{,}5$) y 24,9 % ($h = 1$) de $\sum_i D_i$. En 10 800 rutas al azar, el error de EvalHeur es en promedio 5,9 % de la manipulación (máximo 58 %; el artículo informa 8,66 % y 52,7 % para el umbral 2, Fig. 1), pero solo 0,8 % del total.

4.1. Evaluación exacta

El artículo calcula primero los costos p_{ij} (Algoritmo 2.1) y después resuelve el DP $f(i) = \min_{j>i}\{p_{ij} + f(j)\}$, $f(n) = 0$, donde $f(i)$ es el costo mínimo de manipulación de los clientes $i+1, \ldots, n$ si en i se aplicó la Política 2. El proyecto conserva las dos piezas y pondera cada costo por la posición en que ocurre.

Algoritmo 1 $P(\pi)$: costos p_{ij} (Algoritmo 2.1 adaptado)

Entrada: ruta π con n clientes; α_j, β_j: unidades de entrega y recogida del cliente j; u_b, ℓ_b: tiempos de descargar y cargar una recogida; h_a: tiempo de retirar y recargar una entrega.
Salida: Costos p_{ij} de aplicar la Política 1 en $i{+}1, \ldots, j{-}1$ y la Política 2 en j, $\forall\, i \in \{0, \ldots, n{-}1\}$, $j \in \{i{+}1, \ldots, n\}$.

for $i \leftarrow 0$ **to** $n-1$ **do**
  $\beta' \leftarrow 0$; $\theta \leftarrow 0$ // se aplicó la Pol. 2 en i
  $\alpha' \leftarrow \sum_{m=i+1}^{n} \alpha_{\pi_m}$ // entregas que aún quedan a bordo
  for $j \leftarrow i+1$ **to** n **do**
    $W \leftarrow n - j + 1$ // peso de latencia en la posición j
    $\alpha' \leftarrow \alpha' - \alpha_{\pi_j}$ // entregar, con Pol. 1 o 2
    $b \leftarrow \left(u_b W + \ell_b (W-1)\right) \beta'$ // retirar y recargar las recogidas de la puerta
    if $\beta' + \beta_{\pi_j} > 0$ **then** // costo Pol. 1 hasta aquí + costo Pol. 2
      $p_{ij} \leftarrow \theta + b + h_a\, \alpha'\, (W-1)$
    else
      $p_{ij} \leftarrow \theta + b$ // sin recogidas, la Pol. 2 equivale a la 1
    end if
    $\theta \leftarrow \theta + b$ // Política 1 en j
    $\beta' \leftarrow \beta' + \beta_{\pi_j}$ // recoger como si se usara la Pol. 1
  end for
end for

Cuadro 5: Cambios del Algoritmo 1 respecto del Algoritmo 2.1 del artículo.

Línea	En el artículo (TSPPD-H)	En el proyecto (TRPPD-H$_3$)
5	no existe: cada operación cuesta lo mismo en cualquier posición	$W = n - j + 1$: lo que ocurre en la posición j atrasa a W clientes
7, 13	las recogidas de la puerta cuestan $h_b\beta'$, solo si $\alpha_{\pi_j} > 0$	$(u_b W + \ell_b(W-1))\,\beta'$: se descargan antes de completar la entrega y se recargan después. La condición sobra porque en el TRPPD-H$_3$ $\alpha_i > 0$
9	$p_{ij} = \theta + h_a\alpha' + h_b\beta'$	$p_{ij} = \theta + b + h_a\alpha'(W-1)$: las entregas que quedan se mueven después de entregar a π_j. En $j = n$ la Política 2 no cuesta ($W - 1 = 0$)

Algoritmo 2 EvalExacta(π) (Algoritmo 2.1 y DP (1)–(2))

Entrada: ruta π; datos de la instancia: c_{ij}, α_j, β_j, u_a, ℓ_a, u_b, ℓ_b y Q.
Salida: $\sum_i D_i$ mínimo de la ruta y las políticas s que lo logran; ∞ si π no respeta Q.

1: **if** π no respeta Q **then**
2: **return** ∞
3: **end if**
4: $F \leftarrow \sum_{k=1}^{n} \left[W_k \left(c_{\pi_{k-1}\pi_k} + u_a \alpha_{\pi_k} \right) + (W_k - 1)\, \ell_b \beta_{\pi_k} \right]$ // viaje y operaciones obligatorias
5: calcular p_{ij} con el Algoritmo 1
6: $f(n) \leftarrow 0$
7: **for** $i \leftarrow n-1$ **downto** 0 **do** // $f(i)$: Pol. 2 aplicada en i
8: $f(i) \leftarrow \min_{j \in \{i+1,\ldots,n\}} \{ p_{ij} + f(j) \}$; $sig(i) \leftarrow$ el j del mínimo
9: **end for**
10: $s_{\pi_k} \leftarrow 1 \; \forall k$; $i \leftarrow 0$ // reconstruir las políticas
11: **while** $i < n$ **do**
12: $j \leftarrow sig(i)$
13: **if** $j < n$ **and** $\sum_{m=i+1}^{j} \beta_{\pi_m} > 0$ **then**
14: $s_{\pi_j} \leftarrow 0$ // Política 2 en j
15: **end if**
16: $i \leftarrow j$
17: **end while**
18: **return** $F + f(0)$, s

Cuadro 6: Cambios del Algoritmo 2 respecto del artículo.

Línea	En el artículo (TSPPD-H)	En el proyecto (TRPPD-H$_3$)
4	viaje $\sum_{k=1}^{n+1} c_{\pi_{k-1}\pi_k}$, que en la búsqueda local se actualiza en $O(1)$ con los arcos nuevos y quitados. Las operaciones obligatorias no se evalúan	viaje y operaciones obligatorias con sus pesos, sin el regreso al depósito. Se recalcula en $O(n)$ para cada vecino porque cambian los pesos
resto	DP (1)–(2)	sin cambios: la recursión y la reconstrucción de las políticas son las del artículo

4.2. Evaluación heurística

La Sección 2.2 del artículo estudia un caso especial: todos los clientes con $\bar{\alpha}$ entregas y $\bar{\beta}$ recogidas, y la Política 2 aplicada p veces. Resolviendo ese problema cuadrático obtiene en forma cerrada el número óptimo de Políticas 2, p^* (fórmula (14)), y el largo de cada tramo, k_i^* (fórmula (13)). Con eso prueba cuatro umbrales para decidir cuándo aplicar la Política 2 mientras recorre la ruta, y el mejor es el umbral 2: la cantidad de recogidas a bordo. El artículo no da pseudocódigo de esta evaluación; los Algoritmos 3 y 4 son la lectura que hace el proyecto de esa sección.

Algoritmo 3 $\textsc{Umbral}(i, \pi)$ (Sección 2.2, umbral 2)

Entrada: posición i de la última Política 2 ($i = 0$: depósito); ruta π; h_a, h_b.
Salida: Recogidas junto a la puerta con las que se aplica la próxima Política 2; ∞ si no conviene.

1: $N \leftarrow n - i$ // clientes que faltan
2: $\bar{\alpha} \leftarrow \frac{1}{N}\sum_{m=i+1}^{n} \alpha_{\pi_m}$; $\bar{\beta} \leftarrow \frac{1}{N}\sum_{m=i+1}^{n} \beta_{\pi_m}$ // caso especial con promedios
3: **if** $N < 2$ **or** $\bar{\beta} = 0$ **then**
4: **return** ∞
5: **end if**
6: $r \leftarrow h_b\bar{\beta}/(h_a\bar{\alpha})$
7: $p^* \leftarrow \lfloor (1 - 2r + \sqrt{(2r-1)^2 + 8Nr})/2 \rfloor$ // fórmula (14)
8: **if** $p^* < 2$ **then**
9: **return** ∞ // una sola Pol. 2 al final no ahorra
10: **end if**
11: $k_1^* \leftarrow \max\{1,\ \frac{p^*-1}{2r} + \frac{N}{p^*}\}$ // fórmula (13) con $i = 1$
12: **return** $k_1^*\bar{\beta}$

El Algoritmo 3 usa las fórmulas del artículo tal cual. Esas fórmulas resuelven el caso especial con los costos del TSPPD-H; con los pesos W_k del TRPPD-H$_3$ el caso especial tendría otra solución, porque una operación cuesta menos mientras más tarde ocurre. El proyecto **no volvió a deducir** el umbral: usa la regla del artículo para decidir y mide el costo de esas decisiones con los pesos de latencia (Algoritmo 4). Como el artículo indica, $\bar{\alpha}$ y $\bar{\beta}$ se calculan con los clientes que faltan, así que el umbral se renueva después de cada Política 2.

Algoritmo 4 $\textsc{EvalHeur}(\pi)$ (evaluación lineal de la Sección 2.2)

Entrada: ruta π; datos de la instancia.
Salida: $\sum_i D_i$ de la ruta con las políticas que da el umbral, y esas políticas; ∞ si π no respeta Q.

1: **if** π no respeta Q **then**
2: **return** ∞
3: **end if**
4: $F \leftarrow$ viaje y operaciones obligatorias con sus pesos, como en el Algoritmo 2
5: $\beta' \leftarrow 0$; $\theta \leftarrow 0$; $\tau \leftarrow \textsc{Umbral}(0, \pi)$
6: **for** $j \leftarrow 1$ **to** n **do**
7: $W \leftarrow n - j + 1$
8: $\theta \leftarrow \theta + \big(u_b W + \ell_b(W-1)\big)\,\beta'$ // retirar y recargar las recogidas de la puerta
9: **if** $j < n$ **and** $\beta' + \beta_{\pi_j} > 0$ **and** $\beta' + \beta_{\pi_j} \geq \tau$ **then**
10: $\theta \leftarrow \theta + h_a\,(W-1)\sum_{m=j+1}^{n} \alpha_{\pi_m}$ // Política 2 en j
11: $s_{\pi_j} \leftarrow 0$; $\beta' \leftarrow 0$; $\tau \leftarrow \textsc{Umbral}(j, \pi)$ // nuevo umbral
12: **else**
13: $s_{\pi_j} \leftarrow 1$; $\beta' \leftarrow \beta' + \beta_{\pi_j}$ // Política 1 en j
14: **end if**
15: **end for**
16: **return** $F + \theta$, s

Cuadro 7: Cambios del Algoritmo 4 respecto de la Sección 2.2 del artículo.

Línea	En el artículo (TSPPD-H)	En el proyecto (TRPPD-H$_3$)
4	viaje sin ponderar, actualizado en $O(1)$	parte fija ponderada, en $O(n)$
8, 10	cada decisión cuesta $h_b\beta'$ y $h_a\alpha'$	la misma decisión se mide con los pesos W y $W-1$
regla	umbral 2 con (13)–(14)	sin cambios (Algoritmo 3)

5. Heurística de 2 fases

El artículo construye la solución inicial de sus tres metaheurísticas en dos fases: primero un ciclo TSP por el depósito y los clientes, y después la mejor posición del depósito en ese ciclo (Mosheiov, 1994). El proyecto mantiene la idea, pero cambia la herramienta de la fase 1 y, por el objetivo, el papel de la fase 2.

Cuadro 8: Heurística de 2 fases.

Elemento	Erdoğan et al. (2012) · TSPPD-H	Este proyecto · TRPPD-H_3
Fase 1	Lin–Kernighan de Concorde, usado sin ajustes	vecino más cercano desde cada uno de los $n+1$ nodos, mejorado con 2-opt y Or-opt (tramos de 1 a 3 nodos, en los dos sentidos) con primera mejora; se guarda el ciclo más corto. Admite c asimétrica
Fase 2	entre las posiciones factibles del depósito, la de menor viaje + manipulación óptima	las n rotaciones de cada sentido, evaluadas con EvalExacta; la mejor por sentido
Qué decide la fase 2	la posición del depósito cambia dos arcos del ciclo y la manipulación; con c simétrica, el sentido solo cambia la manipulación	la rotación y el sentido deciden qué clientes se atienden primero, y con eso el peso de todos los arcos
Sentidos	1dir (el de LK) o 2dir (los dos, y se informa el mejor)	siempre los dos: π^{tsp} y π^{inv}
Uso	solo como solución inicial; su tiempo es despreciable	solución inicial y también un método que se informa por sí solo (a lo más 0,1 s)
Calidad	desviación de la solución inicial: 0,9 % (instancias de Battarra, 2dir) y 4,54 % (instancias nuevas, 1dir)	9,4 % sobre la BKS; crece con n: 4,4 % ($n=5$) a 19,1 % ($n=60$). El ciclo minimiza distancia, no latencia

Algoritmo 5 $\mathrm{TSP}(V, c)$ (fase 1)

Entrada: nodos $V = \{0, \ldots, n\}$ (0: depósito); tiempos de viaje c_{ij}.
Salida: Ciclo hamiltoniano C por el depósito y los clientes, de largo $L(C) = \sum_{(i,j)\in C} c_{ij}$.

$C \leftarrow \emptyset$; $L(C) \leftarrow \infty$
for all $v \in V$ **do** // el artículo usa Lin–Kernighan
  $C' \leftarrow$ ciclo del vecino más cercano que parte en v
  repeat // primera mejora
    $mejora \leftarrow$ **false**
    if invertir un tramo de C' (2-opt) reduce $L(C')$ **then**
      aplicar ese 2-opt; $mejora \leftarrow$ **true**
    else if mover un tramo de 1 a 3 nodos, en cualquier sentido (Or-opt), reduce $L(C')$ **then**
      aplicar ese Or-opt; $mejora \leftarrow$ **true**
    end if
  until not $mejora$
  if $L(C') < L(C)$ **then**
    $C \leftarrow C'$
  end if
end for
return C

Algoritmo 6 DosFases(V, c) (TSP + Algoritmo 2.1)

Entrada: instancia del TRPPD-H_3: nodos V, tiempos c_{ij} y los datos de EvalExacta.
Salida: Soluciones iniciales π^{tsp} y π^{inv}, una por sentido del ciclo TSP; la mejor de las dos es la solución de la heurística de 2 fases.

1: $C \leftarrow \text{TSP}(V, c)$ // fase 1
2: $\sigma^{\text{tsp}} \leftarrow$ clientes de C en su orden, sin el depósito
3: $\sigma^{\text{inv}} \leftarrow \sigma^{\text{tsp}}$ en orden inverso
4: **for all** $d \in \{\text{tsp}, \text{inv}\}$ **do** // el artículo: un sentido (1dir) o los dos (2dir)
5: $(\sigma_1, \ldots, \sigma_n) \leftarrow \sigma^d$; $z^d \leftarrow \infty$
6: **for** $r \leftarrow 0$ **to** $n-1$ **do** // fase 2: reubicar el depósito
7: $\pi \leftarrow (\sigma_{r+1}, \ldots, \sigma_n, \sigma_1, \ldots, \sigma_r)$ // el depósito va antes de σ_{r+1}
8: $z \leftarrow$ EvalExacta(π) // $\sum_i D_i$; ∞ si no respeta Q
9: **if** $z < z^d$ **then**
10: $\pi^d \leftarrow \pi$; $z^d \leftarrow z$
11: **end if**
12: **end for**
13: **end for**
14: **return** $\pi^{\text{tsp}}, \pi^{\text{inv}}$

Cuadro 9: Cambios de los Algoritmos 5 y 6 respecto del artículo.

Línea	En el artículo (TSPPD-H)	En el proyecto (TRPPD-H_3)
2–6	Lin–Kernighan	vecino más cercano desde cada nodo, 2-opt y Or-opt: no depende de Concorde. El ciclo puede ser algo más largo que el de LK
4	el sentido de LK (1dir) o los dos (2dir)	siempre los dos
8	viaje + manipulación óptima	$\sum_i D_i$ con políticas óptimas (Algoritmo 2)

En la heurística de 2 fases no hay búsqueda: su solución es la mejor entre π^{tsp} y π^{inv}, con las políticas que da EvalExacta. Como no hay vecinos que evaluar, no tiene versión heurística, ni en el artículo ni en el proyecto.

Ejemplo. En la instancia vigo_n10_id6_h0.1, el ciclo TSP mide 245 y 12 de sus 20 rotaciones (10 por sentido) respetan Q. Con el criterio del artículo, viaje + manipulación, gana la rotación original del ciclo (costo 252,7, $\sum_i D_i = 1534{,}0$). Con $\sum_i D_i$ gana otra rotación, en el sentido inverso y con el depósito entre otros dos clientes (largo 260, $\sum_i D_i = 973{,}9$), que coincide con la mejor solución de ILS exacto. Elegir con el criterio del artículo habría dejado una ruta 57,5 % peor en $\sum_i D_i$.

6. Vecindario y perturbación

ILS e ITS comparten el vecindario y la perturbación. El artículo eligió relocate y 2-opt después de probar también swap (Fig. 3), y fijó $N_{rand} = 0{,}1\,|V_c|$ con un experimento sobre el ILS (Fig. 5). El proyecto usa lo mismo, sin recalibrar los parámetros para el TRPPD-H_3.

Cuadro 10: Vecindario y perturbación.

Elemento	Erdoğan et al. (2012) · TSPPD-H	Este proyecto · TRPPD-H$_3$
Vecindario	relocate ($\lvert V_c\rvert(\lvert V_c\rvert-1)$) y 2-opt ($(\lvert V_c\rvert+1)(\lvert V_c\rvert-2)/2$)	relocate ($n(n-1)$) y 2-opt sobre posiciones $i<j$ ($n(n-1)/2$)
Inversión completa	el 2-opt quita dos arcos no adyacentes del ciclo, así que invertir toda la ruta no es un vecino (el otro sentido se aprovecha con 2dir)	el 2-opt con $i=1$, $j=n$ invierte toda la ruta: es el único movimiento que el proyecto tiene de más ($n(n-1)/2=(n+1)(n-2)/2+1$)
Factibilidad	solo movimientos factibles, sin penalizaciones	igual; la carga solo depende de la ruta y se revisa en $O(n)$
Perturbación	N_{rand} pares (i,j) al azar: relocate de i a un $p\neq i$ al azar si $i=j$, 2-opt entre i y j si no	igual, con $N_{rand}=\max\{1,\mathrm{round}(0{,}1\,n)\}$
Movimientos infactibles	no se detalla	se sortea otro par, hasta 100 intentos por movimiento
Azar	no se informa la semilla	generador mulberry32 con semilla fija (réplicas: semillas $1,\dots,10$); cada corrida se puede repetir

Algoritmo 7 $\textsc{Perturbar}(\pi, N_{rand})$

Entrada: ruta π; N_{rand}: número de movimientos al azar.
Salida: Ruta perturbada que respeta Q.

for $r \leftarrow 1$ **to** N_{rand} **do**
  repeat // a lo más 100 intentos
    elegir $i, j \in \{1,\dots,n\}$ al azar
    if $i = j$ **then**
      elegir $p \in \{1,\dots,n\}\setminus\{i\}$ al azar; $\pi' \leftarrow \textsc{Relocate}(\pi, i, p)$
    else
      $\pi' \leftarrow \textsc{DosOpt}(\pi, \min\{i,j\}, \max\{i,j\})$
    end if
  until π' respeta Q
  if π' respeta Q **then**
    $\pi \leftarrow \pi'$
  end if
end for
return π

7. ILS exacto e ILS heurístico

El ILS del artículo (Algoritmo 4.2) repite N_{iter} veces: tomar la mejor ruta, perturbarla con N_{rand} movimientos al azar, mejorarla con la búsqueda local de mejor mejora en relocate $\cup$ 2-opt, y reemplazar la mejor ruta si se encontró una mejor. El proyecto sigue el mismo esquema; las dos versiones solo difieren en cómo se evalúan los vecinos.

Cuadro 11: ILS.

Elemento	Erdoğan et al. (2012) · TSPPD-H	Este proyecto · TRPPD-H_3
Esquema	Algoritmo 4.2	igual
Búsqueda local	mejor mejora ("Perform Best Move") mientras mejore	igual, con $\sum_i D_i$
Evaluar un vecino	viaje $O(1)$ + manipulación $O(n^2)$ u $O(n)$	parte fija $O(n)$ + manipulación $O(n^2)$ u $O(n)$: mismo orden
Iteraciones	$N_{iter} = 200$ por sentido (probado con 50, 100 y 200)	$N_{iter} = 200$ por sentido
Versión heurística	el mejor vecino se reevalúa con el DP	igual; si su valor exacto no mejora, la búsqueda local termina
Desempeño	el más débil de los tres: 3,85 % (1dir) y 2,68 % (2dir) con evaluación exacta. Con $\|V_c\| \geq 60$ no mejora la solución inicial (Tablas 8–9)	ILS exacto es el mejor de los cinco: 0,01 % sobre la BKS, pero el más lento (43,9 s en promedio, 242 s con $n = 60$). ILS heurístico: 0,13 %, 6,7 s

La diferencia de desempeño puede deberse al punto de partida: en el TSPPD-H la solución inicial de LK ya es buena (0,9 % a 4,5 %), mientras que en el TRPPD-H_3 el ciclo TSP queda lejos de una buena ruta de latencia (9,4 % en promedio, 19 % con $n = 60$) y la búsqueda local tiene mucho que mejorar.

Algoritmo 8 ILS-EXACTO$(\pi^0, N_{iter}, N_{rand})$ (Algoritmo 4.2)

Entrada: solución inicial π^0 de la heurística de 2 fases; N_{iter}: iteraciones; N_{rand}: movimientos al azar por perturbación.

Salida: Mejor ruta encontrada π^* y su $\sum_i D_i$, z^*.

$\pi^* \leftarrow \pi^0$; $z^* \leftarrow$ EVALEXACTA(π^0)
for $it \leftarrow 1$ **to** N_{iter} **do**
  $\pi \leftarrow$ PERTURBAR(π^*, N_{rand}); $z \leftarrow$ EVALEXACTA(π)
  repeat // búsqueda local con mejor mejora
    $\pi' \leftarrow \arg\min_{\sigma \in \mathcal{N}(\pi)}$ EVALEXACTA(σ) // todos los vecinos con el DP
    $z' \leftarrow$ EVALEXACTA(π') // ya calculado al explorar
    $mejora \leftarrow (z' < z)$
    if $mejora$ **then**
      $\pi \leftarrow \pi'$; $z \leftarrow z'$
    end if
  until not $mejora$
  if $z < z^*$ **then** // se conserva la mejor ruta
    $\pi^* \leftarrow \pi$; $z^* \leftarrow z$
  end if
end for
return π^*, z^*

Algoritmo 9 ILS-HEUR$(\pi^0, N_{iter}, N_{rand})$ (Algoritmo 4.2 con evaluación heurística)

Entrada: solución inicial π^0 de la heurística de 2 fases; N_{iter}: iteraciones; N_{rand}: movimientos al azar por perturbación.

Salida: Mejor ruta encontrada π^* y su $\sum_i D_i$ exacto, z^*.

1: $\pi^* \leftarrow \pi^0$; $z^* \leftarrow$ EVALEXACTA(π^0)
2: **for** $it \leftarrow 1$ **to** N_{iter} **do**
3: $\pi \leftarrow$ PERTURBAR(π^*, N_{rand}); $z \leftarrow$ EVALEXACTA(π)
4: **repeat** // búsqueda local con mejor mejora
5: $\pi' \leftarrow \arg\min_{\sigma \in \mathcal{N}(\pi)}$ EVALHEUR(σ) // todos los vecinos con el umbral
6: $z' \leftarrow$ EVALEXACTA(π') // reevaluar solo el vecino elegido
7: $mejora \leftarrow (z' < z)$
8: **if** $mejora$ **then**
9: $\pi \leftarrow \pi'$; $z \leftarrow z'$
10: **end if**
11: **until not** $mejora$
12: **if** $z < z^*$ **then**
13: $\pi^* \leftarrow \pi$; $z^* \leftarrow z$
14: **end if**
15: **end for**
16: **return** π^*, z^*

Cuadro 12: Lo único que distingue a ILS exacto de ILS heurístico.

Línea	ILS exacto (Alg. 8)	ILS heurístico (Alg. 9)
5 / 5	los $\approx 1{,}5\,n^2$ vecinos con EVALEXACTA: $O(n^4)$ por pasada	los mismos vecinos con EVALHEUR: $O(n^3)$ por pasada
6 / 6	sin costo extra	una evaluación exacta por pasada; con $n = 60$, unas 5900 exactas frente a 29 millones heurísticas

8. ITS exacto e ITS heurístico

El ITS del artículo (Algoritmo 4.3) es un ILS que cambia la búsqueda local por una búsqueda tabú (Algoritmo 4.1) de $N^* = \lfloor \sqrt{N_{iter}} \rfloor$ iteraciones, y repite el ciclo perturbación–tabú N^* veces. En cada iteración tabú se aplica el mejor movimiento no tabú, aunque empeore. El proyecto mantiene la estructura y los parámetros; los detalles de la lista tabú son los que cambian.

Cuadro 13: ITS y su búsqueda tabú.

Elemento	Erdoğan et al. (2012) · TSPPD-H	Este proyecto · TRPPD-H$_3$
Iteraciones	$N^* = \lfloor\sqrt{N_{iter}}\rfloor$ externas e internas (7, 10 y 14 para $N_{iter} = 50, 100, 200$)	$N^* = 14$ y 14: 196 iteraciones tabú por sentido
Movimiento	mejor relocate no tabú y mejor 2-opt no tabú; se aplica el mejor de los dos	igual: el mejor de relocate ∪ 2-opt
Atributo tabú	pares (i, j): prohíbe el 2-opt entre los arcos i y j; (i, i) prohíbe reubicar al cliente i (en el Algoritmo 4.1, i es su posición)	2-opt: el par de posiciones (i, j); relocate: el cliente reubicado
Largo de la lista	$0{,}5\,\lvert V_c\rvert$, calibrado en la Fig. 4	$\max\{1, \lfloor 0{,}5\,n\rfloor\}$, sin recalibrar
Aspiración	un movimiento tabú se acepta si mejora *costCurrent*	se acepta si mejora z^b, la mejor ruta de la búsqueda tabú en curso
Lista al empezar	no se detalla	vacía en cada llamada a la búsqueda tabú
Contador	no se reinicia al mejorar	igual: siempre N^* iteraciones
Versión heurística	el movimiento elegido se reevalúa con el DP	igual; la aspiración compara el valor heurístico del vecino con z^b, que es exacto
Desempeño	ITS-1dir 0,83 %, ITS-2dir 0,66 % (exacto)	ITS exacto 0,33 % y 3,7 s; ITS heurístico 0,60 % y 0,67 s. Con $n \geq 40$ no alcanza a converger (1,2 % y 2,0 % con $n = 60$)

Algoritmo 10 ITS-EXACTO$(\pi^0, N_{iter}, N_{rand})$ (Algoritmos 4.3 y 4.1)

Entrada: solución inicial π^0 de la heurística de 2 fases; N_{iter}; N_{rand}: movimientos al azar por perturbación.
Salida: Mejor ruta encontrada π^* y su $\sum_i D_i$, z^*.

$N^* \leftarrow \lfloor\sqrt{N_{iter}}\rfloor$; $L \leftarrow \max\{1, \lfloor n/2\rfloor\}$ // iteraciones y largo de la lista tabú
$\pi^* \leftarrow \pi^0$; $z^* \leftarrow$ EVALEXACTA(π^0)
for $it \leftarrow 1$ **to** N^* **do**
  $\pi \leftarrow$ PERTURBAR(π^*, N_{rand}); $z \leftarrow$ EVALEXACTA(π)
  $\pi^b \leftarrow \pi$; $z^b \leftarrow z$; $T \leftarrow ()$ // búsqueda tabú desde π
  for $t \leftarrow 1$ **to** N^* **do**
    $\mathcal{A} \leftarrow \{\sigma \in \mathcal{N}(\pi) : a(\sigma) \notin T \ \vee$ EVALEXACTA$(\sigma) < z^b\}$ // no tabú o aspiración
    if $\mathcal{A} = \emptyset$ **then**
      break
    end if
    $\pi \leftarrow \arg\min_{\sigma \in \mathcal{A}}$ EVALEXACTA(σ); $z \leftarrow$ EVALEXACTA(π) // se mueve aunque empeore
    agregar $a(\pi)$ al final de T; si $\lvert T\rvert > L$, quitar el más antiguo
    if $z < z^b$ **then**
      $\pi^b \leftarrow \pi$; $z^b \leftarrow z$
    end if
  end for
  if $z^b < z^*$ **then**
    $\pi^* \leftarrow \pi^b$; $z^* \leftarrow z^b$
  end if
end for
return π^*, z^*

Cuadro 14: Cambios del Algoritmo 10 respecto de los Algoritmos 4.3 y 4.1 del artículo.

Línea	En el artículo (TSPPD-H)	En el proyecto (TRPPD-H_3)
5	no se indica si la lista se vacía	se vacía al empezar cada búsqueda tabú
7	atributo (i, i) con la posición i del cliente reubicado; aspiración contra *costCurrent*	atributo (v, v) con el cliente v; aspiración contra z^b

Algoritmo 11 ITS-HEUR$(\pi^0, N_{iter}, N_{rand})$ (Algoritmos 4.3 y 4.1 con evaluación heurística)

Entrada: solución inicial π^0 de la heurística de 2 fases; N_{iter}; N_{rand}: movimientos al azar por perturbación.
Salida: Mejor ruta encontrada π^* y su $\sum_i D_i$ exacto, z^*.

1: $N^* \leftarrow \lfloor\sqrt{N_{iter}}\rfloor$; $L \leftarrow \max\{1, \lfloor n/2 \rfloor\}$
2: $\pi^* \leftarrow \pi^0$; $z^* \leftarrow$ EVALEXACTA(π^0)
3: **for** $it \leftarrow 1$ **to** N^* **do**
4: $\quad\pi \leftarrow$ PERTURBAR(π^*, N_{rand}); $z \leftarrow$ EVALEXACTA(π)
5: $\quad\pi^b \leftarrow \pi$; $z^b \leftarrow z$; $T \leftarrow ()$
6: $\quad$**for** $t \leftarrow 1$ **to** N^* **do**
7: $\quad\quad\mathcal{A} \leftarrow \{\sigma \in \mathcal{N}(\pi) : a(\sigma) \notin T \ \vee$ EVALHEUR$(\sigma) < z^b\}$ // aspiración con el umbral
8: $\quad\quad$**if** $\mathcal{A} = \emptyset$ **then**
9: $\quad\quad\quad$**break**
10: $\quad\quad$**end if**
11: $\quad\quad\pi \leftarrow \arg\min_{\sigma \in \mathcal{A}}$ EVALHEUR(σ) // vecinos con el umbral
12: $\quad\quad z \leftarrow$ EVALEXACTA(π) // reevaluar solo el movimiento elegido
13: $\quad\quad$agregar $a(\pi)$ al final de T; si $|T| > L$, quitar el más antiguo
14: $\quad\quad$**if** $z < z^b$ **then**
15: $\quad\quad\quad\pi^b \leftarrow \pi$; $z^b \leftarrow z$
16: $\quad\quad$**end if**
17: $\quad$**end for**
18: $\quad$**if** $z^b < z^*$ **then**
19: $\quad\quad\pi^* \leftarrow \pi^b$; $z^* \leftarrow z^b$
20: $\quad$**end if**
21: **end for**
22: **return** π^*, z^*

Cuadro 15: Lo único que distingue a ITS exacto de ITS heurístico.

Línea	ITS exacto (Alg. 10)	ITS heurístico (Alg. 11)
7 / 7	aspiración con el valor exacto del vecino	aspiración con el valor heurístico del vecino
11 / 11–12	los vecinos con EVALEXACTA	los vecinos con EVALHEUR; el elegido se reevalúa con EVALEXACTA. En las corridas son siempre $2 + 2 \cdot 14 + 2 \cdot 196 = 422$ evaluaciones exactas

9. Ejecución en los dos sentidos

El artículo comparó correr cada metaheurística desde la solución inicial de LK (1dir) con correrla una vez desde cada sentido del ciclo y quedarse con la mejor (2dir). La versión 2dir mejoró a las tres (Tabla 3), aunque duplica el tiempo. El proyecto usa siempre 2dir, con $N_{iter} = 200$ en cada sentido.

Algoritmo 12 RESOLVER(M, N_{iter}, N_{rand}) (versión 2dir)

Entrada: método $M \in \{\text{ILS-EXACTO}, \text{ILS-HEUR}, \text{ITS-EXACTO}, \text{ITS-HEUR}\}$; N_{iter}; N_{rand}.
Salida: Ruta π^*, políticas s y $\sum_i D_i$ de la mejor solución.

```
(π^tsp, π^inv) ← DOSFASES(V, c)
z* ← ∞
for all π^0 ∈ {π^tsp, π^inv} do                    // una corrida desde cada sentido
    (π, z) ← M(π^0, N_iter, N_rand)
    if z < z* then
        π* ← π;  z* ← z
    end if
end for
(z*, s) ← EVALEXACTA(π*)                           // políticas óptimas de la ruta final
return π*, s, z*
```

Los movimientos al azar usan un generador con semilla, de modo que cada corrida se puede repetir.

10. Resultados

Los números del artículo y del proyecto no se comparan directamente: son problemas, instancias, computadores y referencias distintos. Lo comparable es el comportamiento relativo de los métodos.

Cuadro 16: Artículo: desviación promedio respecto de *Best*, 200 iteraciones, evaluación exacta (Tablas 2, 3 y 5 del artículo; instancias de 20 a 200 clientes).

Método	1dir	2dir	Tiempo 1dir	Con evaluación heurística (2dir)
TS	0,46 %	0,07 %	500,6 s	2,58 % en 117,2 s (Tabla 5)
ILS	3,85 %	2,68 %	1002,6 s	—
ITS	0,83 %	0,66 %	491,8 s	—

Cuadro 17: Proyecto: desviación promedio respecto de la BKS y tiempo promedio por corrida (10 réplicas por instancia, 30 instancias por n; Battarra para $n \le 25$ y Erdoğan para $n \ge 30$). Tiempos de Node.js con 8 procesos en paralelo.

	Desviación (%)					Tiempo (s)			
n	2 fases	ILS-H	ILS-E	ITS-H	ITS-E	ILS-H	ILS-E	ITS-H	ITS-E
5	4,44	0,00	0,00	0,00	0,00	0,002	0,002	0,001	0,001
10	4,69	0,04	0,03	0,04	0,04	0,026	0,041	0,016	0,023
15	6,61	0,00	0,00	0,04	0,03	0,146	0,313	0,048	0,096
20	4,39	0,03	0,00	0,17	0,09	0,363	0,985	0,116	0,286
25	6,96	0,06	0,01	0,23	0,16	1,03	3,54	0,217	0,664
30	9,16	0,14	0,00	0,60	0,25	1,80	6,86	0,361	1,26
40	12,79	0,19	0,02	0,88	0,55	6,14	29,8	0,855	3,88
50	16,27	0,29	0,03	1,47	0,64	15,7	111,5	1,61	9,09
60	19,08	0,43	0,02	2,01	1,17	35,3	242,4	2,83	17,9
Prom.	9,38	0,13	0,01	0,60	0,33	6,72	43,9	0,673	3,70

La BKS de cada instancia es el mínimo entre la mejor solución de las cuatro variantes de Gurobi (límite de 3600 s) y la mejor de todas las réplicas de las cinco heurísticas. Las lecturas principales:

- En los dos trabajos la versión heurística es entre 5 y 9 veces más rápida que la exacta. En el proyecto pierde poca calidad: 0,12 puntos en ILS y 0,27 en ITS.

- El orden de los métodos cambia: en el artículo ILS es el más débil y la búsqueda tabú la más fuerte; en el proyecto ILS exacto es el más preciso e ITS heurístico el más rápido.
- La heurística de 2 fases empeora con n en el TRPPD-H_3 porque el ciclo TSP no considera la latencia; las metaheurísticas corrigen casi toda esa diferencia.

Referencias

Battarra, M., Erdoğan, G., Laporte, G. y Vigo, D. (2010). The traveling salesman problem with pickups, deliveries, and handling costs. *Transportation Science*, 44(3), 383–399.

Erdoğan, G., Battarra, M., Laporte, G. y Vigo, D. (2012). Metaheuristics for the traveling salesman problem with pickups, deliveries and handling costs. *Computers & Operations Research*, 39(5), 1074–1086.

Mosheiov, G. (1994). The travelling salesman problem with pick-up and delivery. *European Journal of Operational Research*, 79(2), 299–310.